PDFMind Sample Document: Computer Science & Data Structures

Chapter 1: Overview of Fundamental Data Structures

A data structure is a specialized format for organizing, processing, retrieving, and storing data. Data structures are designed to organize data to suit a specific purpose so that it can be accessed and worked with in appropriate ways. In computer science, data structures make it easy for users to access, navigate, and sort information efficiently. Basic primitive types include integers, floats, booleans, and characters. Linear data structures include arrays, linked lists, stacks, and queues. Non-linear data structures include trees, graphs, and heaps.

Arrays store elements in contiguous memory locations, allowing O(1) random access by index, but fixed arrays require O(n) time for resizing and insertion at arbitrary positions.

Chapter 2: Trees and Balanced Binary Trees

A Binary Search Tree (BST) is a node-based binary tree data structure where each node has at most two children. For every node X, all elements in the left subtree are strictly smaller than X, and all elements in the right subtree are greater than X. In an ideal balanced BST, search, insertion, and deletion operations take O(log n) time. However, if elements are inserted in sorted order, the BST degenerates into a linear linked list with O(n) worst-case time complexity.

To solve this degeneration issue, self-balancing binary search trees were developed. The most well-known self-balancing trees are AVL Trees and Red-Black Trees.

Chapter 3: In-Depth Study of AVL Trees

An AVL tree is a self-balancing binary search tree named after its two Soviet inventors, Georgy Adelson-Velsky and Evgenii Landis, who published it in their 1962 paper 'An algorithm for the organization of information'. In an AVL tree, the heights of the two child subtrees of any node differ by at most one. If at any time they differ by more than one, rebalancing is done via tree rotations to restore this property.

Lookup, insertion, and deletion all take O(log n) time in both the average and the worst cases. Additions and deletions may require the tree to be rebalanced by one or more tree rotations: Left Rotation (LL), Right Rotation (RR), Left-Right Rotation (LR), and Right-Left Rotation (RL).

The balance factor of a node is defined as the height of its left subtree minus the height of its right subtree: BalanceFactor(N) = Height(LeftSubtree) - Height(RightSubtree). In an AVL tree, the balance factor of every node must be -1, 0, or +1.

Chapter 4: Hash Tables and Collision Resolution

A Hash Table (hash map) is a data structure that implements an associative array abstract data type, a structure that can map keys to values. A hash table uses a hash function to compute an index into an array of buckets or slots, from which the desired value can be found.

Ideally, the hash function will assign each key to a unique bucket, but most hash table designs employ an imperfect hash function, which might cause hash collisions where the hash function generates the same index for more than one key. Such collisions are typically accommodated in two primary ways: Separate Chaining and Open Addressing.

In Separate Chaining, each bucket is independent and holds a linked list or small binary tree of entries with the same index. In Open Addressing (such as Linear Probing, Quadratic Probing, or Double Hashing), all entry records are stored in the bucket array itself. When a collision occurs, alternative cells are probed until an empty cell is found.

Average time complexity for search, insert, and delete is O(1). Worst case with poor distribution is O(n).

Chapter 5: Graph Representation and Traversal

A graph is a non-linear data structure consisting of a finite set of vertices (or nodes) and a set of edges connecting pairs of vertices. Graphs can be directed (digraphs) or undirected, weighted or unweighted, cyclic or acyclic.

Standard graph representations include:
1. Adjacency Matrix: A 2D array of size V x V where V is the number of vertices. Good for dense graphs; lookup is O(1), but memory requirement is O(V^2).
2. Adjacency List: An array of lists where each element lists the neighbors of vertex i. Memory efficient for sparse graphs with space complexity O(V + E).

Fundamental graph traversal algorithms:
- Breadth-First Search (BFS): Uses a Queue and explores neighbor nodes before moving to the next level. Runs in O(V + E) time. Finds the shortest path in unweighted graphs.
- Depth-First Search (DFS): Uses a Stack or recursion, exploring as far as possible along each branch before backtracking. Runs in O(V + E) time. Used in topological sorting, cycle detection, and strongly connected components (Kosaraju or Tarjan algorithms).